

Fiche récapitulative – STA211

Stacking : agrégation par méta-apprentissage

Implémentation Python avec scikit-learn

Vincent Audigier, Ndèye Niang (CNAM)

15 juin 2026

Résumé

Le **stacking** (ou *stacked generalization*) est une méthode d'ensemble qui combine plusieurs modèles de base (*base learners*) via un méta-modèle (*meta-learner*) appris sur les prédictions des modèles de base. Contrairement au bagging ou au boosting, le stacking n'est pas un algorithme unique mais un cadre général permettant de combiner des modèles hétérogènes.

Table des matières

1	Principe du stacking	2
1.1	Architecture générale	2
1.2	Pourquoi ça marche ?	2
1.3	Procédure de génération des données niveau 1	2
2	Implémentation Python avec scikit-learn	2
2.1	StackingClassifier	2
2.2	Paramètres importants	3
2.3	StackingRegressor	3
3	Comparaison avec d'autres méthodes d'ensemble	4
3.1	Voting vs Stacking	4
4	Stratégies avancées	4
4.1	Stacking multi-niveaux	4
4.2	Stacking avec caractéristiques originales	4
4.3	Feature-weighted stacking	4
5	Exercices	4
5.1	Exercice 1 : Stacking manuel	4
5.2	Exercice 2 : Comparaison expérimentale	5
5.3	Exercice 3 : Stacking régression	5
6	QCM	5

Principe du stacking

Architecture générale

Le stacking procède en deux niveaux :

1. **Niveau 0 (base learners)** : K modèles hétérogènes $f_1, \dots, f_K$ sont entraînés indépendamment sur les données d'apprentissage.
2. **Niveau 1 (meta-learner)** : un méta-modèle g est entraîné sur les prédictions des K modèles de base.

Pour une observation x , la prédiction finale est :

$$\hat{y} = g(f_1(x), f_2(x), \dots, f_K(x)).$$

À retenir. La force du stacking réside dans la diversité des modèles de base. Chaque apprenant capture des aspects différents des données ; le méta-modèle apprend à corriger leurs erreurs respectives.

Pourquoi ça marche ?

- **Complémentarité** : des modèles de familles différentes (arbres, réseaux, régression linéaire, SVM) génèrent des prédictions diversifiées.
- **Correction des biais** : le méta-modèle apprend les motifs d'erreur des modèles de base.
- **Réduction de variance** : la combinaison de multiples estimateurs lisse les prédictions individuelles.

Procédure de génération des données niveau 1

Pour éviter le surapprentissage, les prédictions niveau 1 sont générées par validation croisée (souvent 5-fold) :

1. Diviser les données d'apprentissage D en V plis.
2. Pour chaque pli v : entraîner chaque f_k sur $D \setminus D_v$, prédire sur D_v .
3. Les prédictions hors-pli pour tous les K modèles forment la matrice Z ($N \times K$) d'entrée du méta-modèle.
4. Entraîner chaque f_k sur D complet (optionnel).
5. Entraîner g sur (Z, y) .

Implémentation Python avec scikit-learn

StackingClassifier

Scikit-learn propose `StackingClassifier` et `StackingRegressor` depuis la version 0.22.

```

1 from sklearn.ensemble import StackingClassifier
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.svm import SVC
5 from sklearn.neighbors import KNeighborsClassifier
6 from sklearn.datasets import load_breast_cancer
7 from sklearn.model_selection import cross_val_score
8 import numpy as np
9
10 # Données
11 X, y = load_breast_cancer(return_X_y=True)
12
```

```

13 # Modeles de base heterogenes
14 base_learners = [
15     ('lr', LogisticRegression(max_iter=5000)),
16     ('knn', KNeighborsClassifier(n_neighbors=5)),
17     ('svm', SVC(probability=True, random_state=0))
18 ]
19
20 # Meta-modele (regression logistique)
21 meta_learner = LogisticRegression(max_iter=5000)
22
23 # Stacking
24 stack = StackingClassifier(
25     estimators=base_learners,
26     final_estimator=meta_learner,
27     cv=5 # validation croisee 5-fold pour Z
28 )
29
30 # Evaluation
31 scores = cross_val_score(stack, X, y, cv=5)
32 print(f"Stacking: {scores.mean():.3f} +/- {scores.std():.3f}")

```

Paramètres importants

- `estimators` : liste de tuples (nom, modèle).
- `final_estimator` : le méta-modèle (défaut : régression logistique pour classification, Ridge pour régression).
- `cv` : nombre de plis pour générer les prédictions niveau 1 (défaut : 5).
- `stack_method` : 'auto' (probabilités si disponibles, sinon prédictions brutes), 'predict_proba', 'predict'.
- `passthrough` : si True, ajoute les caractéristiques originales X aux prédictions niveau 1.

StackingRegressor

```

1 from sklearn.ensemble import StackingRegressor
2 from sklearn.linear_model import Ridge, Lasso
3 from sklearn.tree import DecisionTreeRegressor
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.datasets import load_diabetes
6
7 X, y = load_diabetes(return_X_y=True)
8
9 base_learners = [
10     ('ridge', Ridge(alpha=1.0)),
11     ('lasso', Lasso(alpha=0.1)),
12     ('rf', RandomForestRegressor(n_estimators=100,
13                                 random_state=0))
14 ]
15
16 stack = StackingRegressor(
17     estimators=base_learners,
18     final_estimator=Ridge(alpha=1.0),
19     cv=5
20 )
21
22 scores = cross_val_score(stack, X, y, cv=5,
23                           scoring='neg_mean_squared_error')
24 print(f"RMSE: {np.sqrt(-scores.mean()):.3f}")

```

Comparaison avec d'autres méthodes d'ensemble

Méthode	Base learners	Entraînement	Réduction
Bagging	même type	parallèle	variance
Boosting	même type	séquentiel	biais + variance
Voting	hétérogènes	indépendant	variance
Stacking	hétérogènes	indépendant	biais + variance

Voting vs Stacking

Le `VotingClassifier` combine les prédictions par vote majoritaire (classification) ou moyenne (régression). Le stacking apprend *comment* combiner les prédictions, ce qui offre plus de flexibilité mais risque le surapprentissage.

Stratégies avancées

Stacking multi-niveaux

On peut empiler plusieurs couches de méta-modèles :

Niveau 0 $\rightarrow$ Niveau 1 $\rightarrow$ Niveau 2

Attention au surapprentissage : à chaque niveau, les prédictions doivent être générées par CV.

Stacking avec caractéristiques originales

Avec `passthrough=True`, le méta-modèle reçoit les prédictions **et** les variables originales X , ce qui lui permet d'apprendre des corrections contextuelles.

Feature-weighted stacking

On peut pondérer chaque modèle de base par sa performance estimée sur un pli de validation plutôt que de laisser le méta-modèle apprendre tous les poids.

Exercices

Exercice 1 : Stacking manuel

Implémenter à la main un stacking avec 3 classifieurs (KNN, DecisionTree, SVM) et une régression logistique comme méta-modèle, sans utiliser `StackingClassifier` :

```

1 from sklearn.model_selection import KFold
2 import numpy as np
3
4 def stacking_manual(X, y, base_learners, meta_learner, n_folds=5):
5     kf = KFold(n_splits=n_folds, shuffle=True, random_state=0)
6     Z = np.zeros((len(y), len(base_learners)))
7
8     for train_idx, val_idx in kf.split(X):
9         X_train, X_val = X[train_idx], X[val_idx]
10        y_train, y_val = y[train_idx], y[val_idx]
11
12        for k, (nom, modele) in enumerate(base_learners):
13            modele.fit(X_train, y_train)
14            Z[val_idx, k] = modele.predict_proba(X_val)[:, 1]
```

```
15
16     # Entraîner le meta-modele sur Z
17     meta_learner.fit(Z, y)
18
19     # Re-entraîner les base learners sur tout X
20     for _, modele in base_learners:
21         modele.fit(X, y)
22
23     return meta_learner, base_learners
```

Exercice 2 : Comparaison expérimentale

Comparer les performances de : (a) chaque modèle de base seul, (b) VotingClassifier, (c) StackingClassifier sur un jeu de données de classification (breast cancer, iris, ou un jeu synthétique). Interpréter les résultats.

Exercice 3 : Stacking régression

Utiliser **StackingRegressor** pour combiner Ridge, Lasso et RandomForest sur les données **diabetes**. Comparer le RMSE avec chaque modèle individuel.

QCM

1. Quelle méthode génère les données d'entrée du méta-modèle ?
 - A. Un échantillonnage bootstrap
 - B. Une validation croisée
 - C. Un tirage aléatoire simple
 - D. Une ACP préalable
2. Quel est l'avantage principal du stacking par rapport au VotingClassifier ?
 - A. Il est plus rapide à l'entraînement
 - B. Il apprend à combiner les prédictions
 - C. Il utilise moins de mémoire
 - D. Il ne nécessite pas de base learners
3. Quel paramètre permet d'ajouter les variables originales au méta-modèle ?
 - A. `add_features`
 - B. `passthrough`
 - C. `original_X`
 - D. `include_X`
4. Le stacking réduit principalement :
 - A. Le biais uniquement
 - B. La variance uniquement
 - C. Le biais et la variance
 - D. Ni le biais ni la variance
5. Quelle est la principale difficulté du stacking multi-niveaux ?
 - A. Le temps d'exécution
 - B. Le surapprentissage
 - C. La parallélisation
 - D. Le choix des hyper-paramètres
6. Que fait `stack_method='predict_proba'` ?
 - A. Utilise les probabilités des modèles de base
 - B. Utilise les classes prédites

- C. Utilise les écarts-types
 - D. Utilise les résidus
- 7.** Dans le stacking manuel, pourquoi ré-entraîner les base learners sur tout X après la CV ?
- A. Pour économiser de la mémoire
 - B. Pour améliorer leurs performances
 - C. Pour réduire le biais
 - D. Ce n'est pas nécessaire
- 8.** Quel estimateur final est utilisé par défaut dans `StackingClassifier` ?
- A. Régression logistique
 - B. `DecisionTreeClassifier`
 - C. SVC
 - D. `RandomForestClassifier`
- 9.** Pourquoi les prédictions niveau 1 doivent-elles être générées par CV ?
- A. Pour accélérer l'entraînement
 - B. Pour éviter le surapprentissage du méta-modèle
 - C. Pour garantir la convergence
 - D. Pour équilibrer les classes
- 10.** Quel type de modèles de base le stacking peut-il utiliser ?
- A. Uniquement des arbres
 - B. Uniquement des modèles linéaires
 - C. Des modèles hétérogènes
 - D. Uniquement des SVM

Corrigé du QCM

1-B, 2-B, 3-B, 4-C, 5-B, 6-A, 7-B, 8-A, 9-B, 10-C.